

The background is a blurred image of a computer monitor. The screen shows a dark-themed code editor with some text in green and blue. At the bottom, a Windows taskbar is visible with various application icons like Chrome, Photoshop, and others.

Introduction To Programming Using C++ Level 2

Exception Handling (Try- Catch)



Introduction To Programming Using C++

Level 2

Exception Handling (Try-Catch)

Definition

Exception Handling

 Exception Handling is a mechanism in C++ used to handle unexpected errors that occur during program execution.

 It allows the program to detect an exception and handle it instead of terminating unexpectedly.

 Example: trying to access an element outside the valid range of a `vector`.

Explanation

``try``

 The ``try`` block contains code that may throw an exception.

``catch``

 The ``catch`` block contains the code that handles the exception if one occurs.

``catch(...)``

 The ``...`` means that the ``catch`` block can catch an exception of any type.

``at()``

 ``vector::at()`` checks whether the requested index is within the valid range.

 If the index is invalid, ``at()`` throws a ``std::out_of_range`` exception.

Procedures

1. Identify code that may throw an exception.
2. Place that code inside a `try` block.
3. Use `catch` to handle the exception.
4. Use `at()` instead of `[]` when you want `vector` bounds checking.
5. Handle the exception appropriately.

💡 Example

```
int main()
{
    vector<int> num = { 1, 2, 3, 4, 5 };
    try
    {
        cout << num.at(5);    // Accessing a non-existing element
    }
    catch (...)
    {
        cout << "Out of bounds";
    }
    return 0;
}
```

➡ Result
Out of bounds

✈ Note

- ◆ `catch(...)` can catch an exception of any type.
- ◆ In larger programs, it is generally better to catch a specific exception type when possible.
- ◆ For example:

```
catch (const out_of_range& e)
{
    cout << e.what();
}
```
- ◆ `std::out_of_range` is the exception type commonly thrown by `vector::at()` when the index is invalid.

⊘ Warning

- ◆ Do not use exceptions unnecessarily for normal program flow.
- ◆ Exception handling has runtime overhead when exceptions are actually thrown.
- ◆ `[]` does not perform bounds checking for `vector`; an invalid index results in undefined behavior rather than a catchable `out_of_range` exception.



Comparison

▣ `at()`

- ✓ Performs bounds checking.
- ✓ Throws `std::out\_of\_range` when the index is invalid.
- ✓ Can be handled using `try` - `catch`.

▤ `operator[]`

- ✓ Does not perform bounds checking.
- ✓ Invalid index results in undefined behavior.
- ✓ Does not throw `std::out\_of\_range` for an invalid index.

✦ Summary

- ✓ ``try`` contains code that may throw an exception.
- ✓ ``catch`` handles the exception.
- ✓ ``catch(...)`` can catch exceptions of any type.
- ✓ ``vector::at()`` checks the index and can throw ``std::out_of_range``.
- ✓ ``vector::operator[]`` does not perform bounds checking.
- ✓ Catching a specific exception type is generally preferable when possible.

Thank you 😊